PYTHON SUMMER PLAN · DAY 04

Properties, classmethods, staticmethods

Guard your attributes. Provide alternate constructors. Group helpers without dragging self around.

Thursday, May 14 · 75 minutes

Yuvraj Ruparel

RECAP · YESTERDAY

Where we left off

shapes.py has Shape, Circle, Box. The physics sandbox runs. But the classes will accept any input — Circle(radius=-3) builds happily, then breaks math later. Today we add the guardrails decorators give you for free.

VALIDATED ATTRIBUTES

@property setter

Make assignment look free but actually run a check. Negative radius? raise ValueError before it corrupts the sim.

ALTERNATE CONSTRUCTORS

@classmethod

Build a Circle a different way. Circle.unit() returns a unit circle. Box.square(s) returns a square.

CLASS-RELATED HELPERS

@staticmethod

A function that lives in the class because it belongs there, but doesn't touch self or cls. Pure utility.

Three decorators. Same goal — make the class API cleaner and harder to misuse.

The problem with raw attributes

BEFORE

No guard

```
class Circle(Shape):
    def __init__(self, pos, vel, radius):
        super().__init__(pos, vel)
        self.radius = radius

c = Circle(p, v, radius=-3)    # builds!
print(c.area)          # 28.27, garbage
```

Anything goes in. Bad data lives inside the object until something blows up downstream.

AFTER

Guarded by a setter

```
@property
def radius(self):  # getter
    return self._radius

@radius.setter
def radius(self, value):
    if value <= 0:
        raise ValueError("radius > 0")
    self._radius = value
```

Same usage — c.radius = 5 — but bad values get caught at the boundary.

How @property + @setter works

```
class Circle(Shape):
    def __init__(self, pos, vel, radius):
        super().__init__(pos, vel)
        self.radius = radius   # triggers setter

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError("radius must be numeric")
        if value <= 0:
            raise ValueError("radius must be > 0")
        self._radius = value
```

Four things to internalize

- **The underscore convention.**

  _radius is the storage. radius is the public access. Outside code only ever touches the public name.

- **@property = the getter.**

  Defined first. The name you use here (radius) becomes the attribute name everyone reads.

- **@radius.setter = the writer.**

  Same method name. The decorator chains onto the property object created above.

- **__init__ triggers the setter.**

  self.radius = radius isn't a raw assign — Python routes it through your setter. Validation runs on construction too.

Same syntax, real protection

```
c = Circle(pos, vel, radius=0.5)     # OK
print(c.radius)                      # reads via getter
c.radius = 0.8                       # OK, setter checks pass

c.radius = -3                        # ValueError
c.radius = "big"                      # TypeError
c = Circle(pos, vel, radius=-3)      # ValueError at build
```

OUTPUT

```
>>> c.radius = -3
Traceback (most recent call last):
  ...
ValueError: radius must be > 0
```

Why this matters

Bad data dies at the boundary. The setter is the only door in.

Downstream code can trust that any Circle it gets has a valid radius. No defensive checks scattered across collision math, no "how did this Circle have radius=0" mysteries at 2am.

Same syntax as before — c.radius = 5. Users of the class never see the difference. That's the point: invisible enforcement.

@classmethod — alternate constructors

__init__ is one way to build an object. classmethods give you more, each with its own name and defaults.

```
class Box(Shape):
    def __init__(self, pos, vel, width, height):
        ...

    @classmethod
    def square(cls, pos, vel, side):
        return cls(pos, vel, side, side)

    @classmethod
    def at_rest(cls, pos, w, h):
        return cls(pos, Vector3D(0, 0, 0), w, h)

# usage
b1 = Box.square(pos, vel, side=1.0)
b2 = Box.at_rest(pos, 2, 1)
```

Why cls, not self

There's no instance yet — we're building one. cls is the class itself (Box, or any subclass that calls .square).

cls(...) calls __init__, which runs setters, which validate. You get all your guards for free.

Subclass-safe

If FancyBox(Box) inherits .square and calls FancyBox.square(...), cls is FancyBox. You get a FancyBox, not a plain Box. Hard-coding Box(...) would lose that.

@staticmethod — namespaced helpers

A regular function that lives in the class because it belongs there. Doesn't touch self or cls.

```python
class Shape:
    ...

    @staticmethod
    def collision_pair(a, b):
        """Return True if either reports collision."""
        return a.collides_with(b) or b.collides_with(a)

    @staticmethod
    def clamp(value, lo, hi):
        return max(lo, min(hi, value))

# usage
if Shape.collision_pair(a, b):
    resolve(a, b)
```

How to choose

Needs self?

→ *Regular method*

Needs cls (alt constructor)?

→ *@classmethod*

Needs neither — just data in/out?

→ *@staticmethod*

The three decorators at a glance

DECORATOR	FIRST ARG	PURPOSE	CALLED AS
@property	self	*Computed / guarded attribute read.*	obj.attr
@x.setter	self, val	*Validate writes to property x.*	obj.attr = v
@classmethod	cls	*Alternate constructor or class-level state.*	Class.method(...)
@staticmethod	(none)	*Helper that belongs in this namespace.*	Class.method(...)

Takeaway

Most methods are regular. Reach for @property when the attribute needs validation. Reach for @classmethod when you want a named alternative to __init__. @staticmethod is rare.

TODAY'S EXERCISE · 75 MIN

Upgrade shapes.py

Add validation to Circle and Box. Add factory constructors. Group one or two utilities as staticmethods. The sandbox from yesterday should still run unchanged.

SPEC

Validation (setters)

Circle: radius > 0 (numeric)

Box: width > 0 and height > 0

Shape: position must be a Vector3D

Raise ValueError or TypeError as appropriate.

Factory constructors

Circle.unit(pos) → radius=1.0

Box.square(pos, vel, side)

Circle.at_rest(pos, radius) — zero velocity

All return cls(...), not Circle(...) / Box(...).

Static helpers

Shape.collision_pair(a, b)

Shape.clamp(v, lo, hi) — useful for wall bounds

Keep it small: only add helpers that genuinely belong to Shape.

Tips & gotchas

Don't shadow the property name

Storage goes in self._radius, not self.radius. If you write self.radius = value inside the setter, you'll hit infinite recursion — the setter calls itself.

Setters run during __init__

self.radius = radius inside __init__ triggers your setter. If validation raises, the constructor raises. That's the point — no half-built objects with bad state.

Use cls, never the hardcoded class name

return cls(...) inside a classmethod so subclasses get their own type back. Box.square called from FancyBox should return a FancyBox.

@property without @setter = read-only

Skip the setter and the attribute becomes read-only. magnitude on Vector3D is already this — that's why v.magnitude = 5 raises AttributeError.

W R A P - U P · W H A T T H I S F E E D S

shapes.py is now defensively built

Yesterday the classes worked. Today they refuse to be misused. Tomorrow we handle the cases where things still go wrong.

F R I D A Y

Exceptions

try / except / raise, EAFP style, custom exception classes. Wrap your validators in clean error types. Fix vector3d.py normalize() zero-magnitude bug.

S A T U R D A Y

Spring-Mass-Damper

SMD parameters (m, k, c) all need validation: must be positive. Damping ratio $\zeta = c / (2\sqrt{(mk)})$ — a perfect @property.

L A T E R

Patterns reused

Every class from here uses these decorators. Validation at the boundary becomes a habit, not a feature.

Once shapes.py refuses bad input cleanly, drop the file back here.